

Week 4: The Birth and Death Process

What is the Birth and Death Process?

Most queueing processes are analysed through what is called a **birth and death process** — a probabilistic model that describes how the number of customers in a system changes over time.

The terminology is borrowed from population biology, where "birth" adds a member to the population and "death" removes one. In queueing systems:

- **Birth** = the **arrival** of a new customer into the system.
- **Death** = the **departure** of a served customer from the system.

So a queue grows by "births" and shrinks by "deaths." The number of customers in the system at any moment is treated as the current "population."

Formal Description

Let $C(t)$ denote the state of the system at time t (for $t \geq 0$). Specifically, $C(t)$ is **the number of customers in the queueing system at time t** .

The birth and death process describes how $C(t)$ changes — i.e. how the system transitions from one state to another over time — using probability distributions.

Real-world example: Imagine a bank at 9:00 AM. If $C(9:00) = 4$, four customers are in the system. A minute later a new arrival pushes the state to $C(9:01) = 5$ (a "birth"). Two minutes later a customer finishes being served and leaves, dropping the state to $C(9:03) = 4$ (a "death"). The birth and death process is the mathematical description of this back-and-forth.

Assumptions in the Birth and Death Process

The classical birth and death process rests on three key assumptions:

1. Arrivals (Births)

The probability distribution of the remaining time until the next arrival is **exponential** with parameter λ_n (where $n = 0, 1, 2, 3, \dots$), and $n = C(t)$ is the current state.

The use of subscript n means the arrival rate can depend on the current state. For instance, a queue might attract fewer customers when it's already long (because they balk). In the simplest M/M/1 model, however, $\lambda_n = \lambda$ for all n — a constant arrival rate.

2. Departures (Deaths)

The probability distribution of the remaining time until the next service completion is **exponential** with parameter μ_n (where $n = 0, 1, 2, 3, \dots$), and again $n = C(t)$.

Likewise, μ_n may depend on the state — for example, a multi-server system serves customers faster when more servers are busy. In the simplest case, $\mu_n = \mu$ for all $n \geq 1$ (and $\mu_0 = 0$, since you can't serve from an empty system).

3. Independence

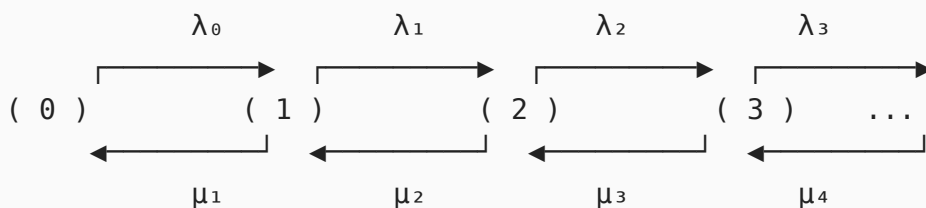
The random variable representing the birth process and the one representing the death process are **mutually independent**. Whatever determines when the next customer arrives has nothing to do with when the current customer finishes.

Why Exponential?

The exponential distribution has a special property called **memorylessness**: the probability of the next event happening in the next minute is the same regardless of how long you've already been waiting. This makes the math much cleaner. It's also a reasonable model for "random" events like phone calls arriving or accidents occurring.

State Diagram

A birth-and-death process is often pictured as a **state transition diagram**, where each circle is a possible value of $C(t)$ and arrows show the transitions:



- **Forward arrows** (rates $\lambda_0, \lambda_1, \lambda_2, \dots$) represent **arrivals** moving the system to a higher state.
- **Backward arrows** (rates $\mu_1, \mu_2, \mu_3, \dots$) represent **departures** returning the system to a lower state.
- Movement is **only between adjacent states** — the system cannot jump from state 1 to state 3 directly, because births and deaths happen one at a time.

This is essentially a **continuous-time Markov chain** — the next state depends only on the current state, not on the history of how we got there.

Birth-Death Process Probabilities

For the simplest birth-and-death queueing model (M/M/1, with constant λ and μ), the long-run probability of finding **exactly** n **customers** in the system is:

$$P_n = (1 - \rho)\rho^n$$

Where $\rho = \lambda/\mu$ is the server utilisation (from Week 2).

What This Formula Tells Us

- P_n is the **steady-state probability** — the long-run fraction of time the system has exactly n customers.
- The probabilities form a **geometric distribution** — each extra customer is less likely than having one fewer.
- $P_0 = 1 - \rho$ — the probability the system is empty equals the probability the server is idle. (This matches Week 2.)
- The formula only works when $\rho < 1$ (stable system). If $\rho \geq 1$, no steady state exists — the queue grows without bound.

Why It Works (Intuition)

In steady state, the rate of entering state n must equal the rate of leaving state n (a "balance equation"). Solving these balance equations across all states yields the geometric form above. You can verify that the probabilities sum to 1:

$$\sum_{n=0} P_n = (1 - \rho) \sum_{n=0} \rho^n = (1 - \rho) \cdot \frac{1}{1 - \rho} = 1 \checkmark$$

Worked Example 1: Bank Operation

In a bank, the arrival rate is **4 customers per hour** and the service rate is **9 customers per hour**. Find the probability that exactly 3 customers are in the system, and state whether the system is stable.

Given

$$\lambda = 4, \quad \mu = 9$$

$$\rho = \frac{\lambda}{\mu} = \frac{4}{9} \approx 0.4444$$

Stability Check

Since $\rho \approx 0.44 < 1$, the system is **stable**. A steady state exists, so the formula applies.

Calculation

$$P_3 = (1 - \rho)\rho^3 = (1 - 0.4444)(0.4444)^3$$

$$P_3 = 0.5556 \times 0.0878 \approx 0.0488$$

Interpretation

About **4.88%** of the time, exactly 3 customers are in the system. In an 8-hour business day, that's roughly 23 minutes spent with exactly 3 customers present.

Worked Example 2: 2-Customers-per-Minute System

The arrival rate is **2 customers per minute** and the service rate is **5 customers per minute**. Find the probability of having:

a) 0 customers b) 1 customer c) 2 customers d) State whether or not the system is stable.

Given

$$\rho = \frac{\lambda}{\mu} = \frac{2}{5} = 0.4 = 40\%$$

Stability Check

$\rho = 0.4 < 1$, so the system is **stable**.

Calculations

a) Zero customers (P_0)

$$P_0 = (1 - 0.4)(0.4)^0 = 0.6 \times 1 = 0.6$$

The system is empty **60% of the time** — meaning the server is idle 60% of the time.

b) One customer (P_1)

$$P_1 = (1 - 0.4)(0.4)^1 = 0.6 \times 0.4 = 0.24$$

There is exactly one customer (being served, no queue) **24% of the time**.

c) Two customers (P_2)

$$P_2 = (1 - 0.4)(0.4)^2 = 0.6 \times 0.16 = 0.096$$

Exactly two customers (one being served, one waiting) is present **9.6% of the time**.

Sanity Check

Notice the probabilities are decreasing rapidly: 0.6, 0.24, 0.096, ... — this is the geometric pattern. Each successive state is $\rho = 0.4$ times as likely as the previous one.

Self-Check Counter-Example (From Notes)

Suppose instead $\lambda = 10$ customers/hr and $\mu = 8$ customers/hr:

$$\rho = \frac{10}{8} = 1.25$$

Since $\rho > 1$, the system is **not stable** — customers arrive faster than they can be served, so the queue grows without bound and no steady-state probability P_n exists. The formula cannot be applied.

Worked Example 3: TV Repairer

A TV repairer receives **5 jobs per hour** and serves **7 customers per hour**. Find the probability that:

a) The system is empty. b) There are exactly 2 jobs in the system. c) There are at least 3 jobs in the system.

Given

$$\lambda = 5, \quad \mu = 7, \quad \rho = \frac{5}{7} \approx 0.7143$$

Since $\rho < 1$, the system is stable.

Calculations

a) System is empty (P_0)

$$P_0 = (1 - 0.7143)(0.7143)^0 = 0.2857 \times 1 = 0.2857$$

The repairer is idle about **28.57%** of the time.

b) Exactly 2 jobs (P_2)

$$P_2 = (1 - 0.7143)(0.7143)^2 = 0.2857 \times 0.5102 \approx 0.1458$$

Two jobs are present about **14.58%** of the time.

c) At least 3 jobs ($P_{n \geq 3}$)

This is a classic use of the **complement rule**: instead of computing $P_3 + P_4 + P_5 + \dots$ (infinitely many terms), we use:

$$P_{n \geq 3} = 1 - P(n < 3) = 1 - (P_0 + P_1 + P_2)$$

First find P_1 :

$$P_1 = (1 - 0.7143)(0.7143)^1 = 0.2857 \times 0.7143 \approx 0.2041$$

Now sum:

$$P_0 + P_1 + P_2 = 0.2857 + 0.2041 + 0.1458 = 0.6356$$

Therefore:

$$P_{n \geq 3} = 1 - 0.6356 = 0.3644$$

Interpretation

About **36.44%** of the time, the repairer has 3 or more jobs piling up. This is a meaningful figure — if jobs piling up causes customer dissatisfaction, the repairer might consider hiring help or speeding up service.

Useful Related Probability Identities

When working with birth-and-death problems, two complementary patterns come up often:

Probability of "at least k " customers

$$P_{n \geq k} = \rho^k$$

This is a shortcut that follows directly from the formula. Let's verify with the TV repairer example:

$$P_{n \geq 3} = \rho^3 = (0.7143)^3 \approx 0.3644 \checkmark$$

Same answer, much less arithmetic!

Probability of "at most k " customers

$$P_{n \leq k} = 1 - \rho^{k+1}$$

For example, $P_{n2} = 1 - \rho^3 = 1 - 0.3644 = 0.6356$ ✓ (matches what we computed above).

Why this works: $P_{n \geq k} = \rho^k$ because each step from one state to the next contributes a factor of ρ . The probability of being "at least k " is essentially the probability of reaching state k , which requires k successive "births" — each with probability ρ of dominating a "death."

Why Birth-and-Death Matters for Modelling

The birth-and-death process is the **theoretical backbone** of basic queueing models. It is important because:

1. **It justifies the formulas of Week 2.** The expressions for L_s , L_q , W_s , W_q all come from solving the balance equations of a birth-and-death process.
2. **It generalises naturally.** By letting λ_n and μ_n depend on n , you can model:
 - **Multi-server queues** (μ_n grows with n up to the number of servers).
 - **Finite-capacity systems** ($\lambda_n = 0$ when n reaches the maximum).
 - **Balking and reneging** (λ_n decreases as the queue grows).
 - **Population dynamics**, epidemic spread, and chemical reactions outside queueing entirely.
3. **It is the foundation of Markov chain modelling** — one of the most widely used techniques in operations research, finance, biology, and machine learning.

Real-world example beyond queues: Epidemiologists use birth-and-death processes to model disease spread. "Births" become new infections; "deaths" become recoveries. The math is identical to a queue — only the labels change.